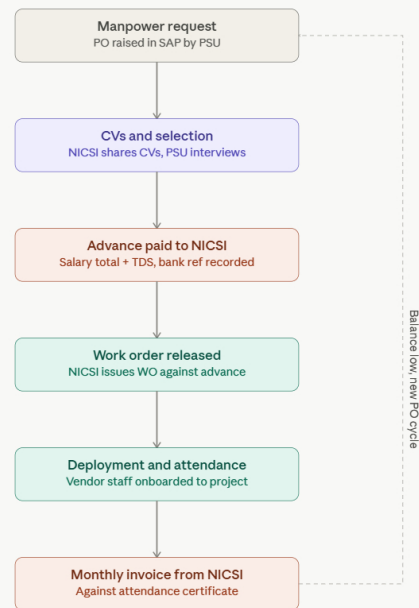


I have a requirement to make an HRMS and vendor management kind of system. The situation is like this: one PSU company hires vendor employees through NICS, and they want a way to manage purchase orders, Work orders, Advance payments, vendor employee details, and attendance. It happens like this: Suppose there are multiple projects in the company and there is a requirement for more manpower, so the project manager can raise a purchase order request through SAP, and the purchase department will handle the purchase order process; we only need to record and track the Purchase order number(PO No.) and other basic PO details, the NICS will

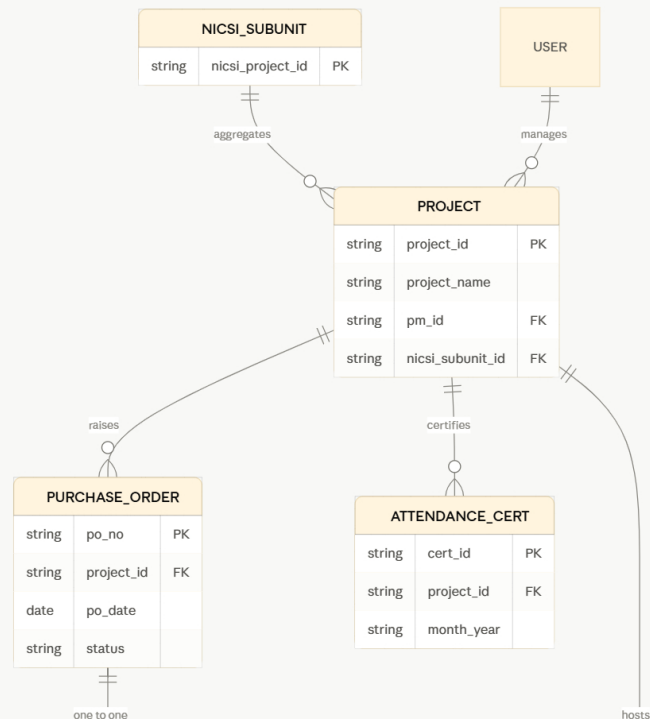
Show more

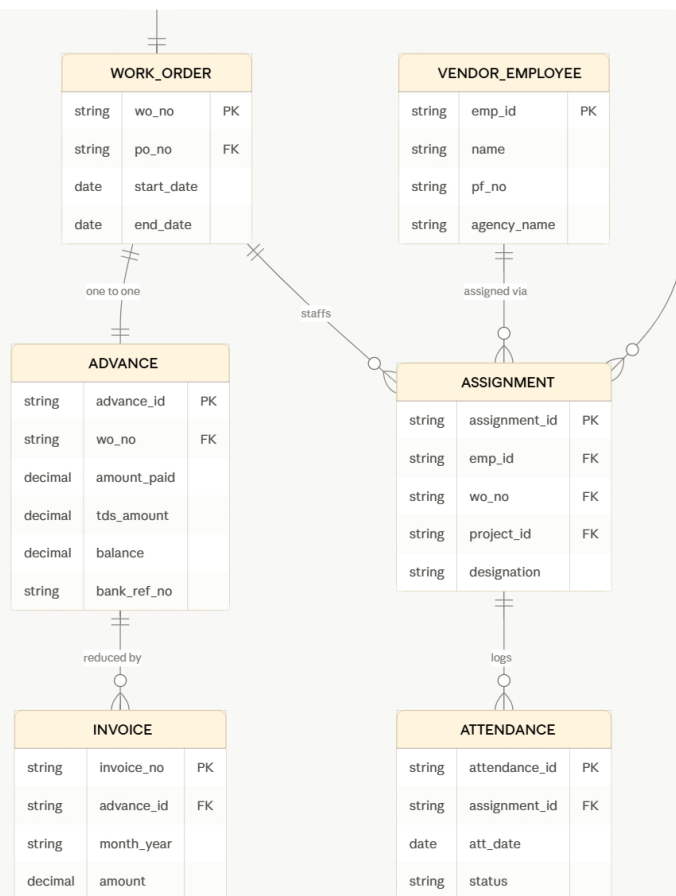
Architected comprehensive system design encompassing data models and workflows



This is the recurring lifecycle for one project: each time the advance runs low and manpower is still needed, it loops back into a fresh PO → WO → advance cycle, while everything stays tagged to the same PSU project ID.

Now let's look at how the data itself should be structured underneath this flow.





A few design decisions worth calling out before the full write-up:

The NICS1-subunit vs PSU-project split is the trickiest part of this model. NICS1 only issues one project ID per PSU subunit, but you want multiple internal project IDs. So `NICS1_SUBUNIT` sits above `PROJECT` as a one-to-many parent — every PSU project rolls up to the single NICS1 ID for subunit-level advance totals, while each PSU project still gets its own PO/WO/advance trail for project-level tracking.

PO ↔ WO is a strict 1:1, and Advance is 1:1 with WO — so the "advance" really lives at the WO level, and a project accumulates advance history across multiple PO→WO→Advance triples over its lifetime (your 6-month renewal example).

Employees are modeled as an assignment, not a direct link — `VENDOR_EMPLOYEE` master data is separate from `ASSIGNMENT`, because the same person can move between work orders/projects over time, and attendance needs to trace back to a specific WO/advance so consumption is attributed correctly.

Here's the full design doc with workflow states, business rules, dashboard specs, and an architecture/tech recommendation:

Created a file, read a file

The doc covers the full data model, workflow states, roles, dashboard specs, and architecture recommendations. A few things worth deciding before you start building, flagged at the end of the doc:

- Whether CV shortlisting deserves its own tracked entity or stays informal
- Whether advance entries need a maker-checker approval given how sensitive bank refs/TDS are
- What triggers a "low balance" alert (percentage vs fixed amount)
- Any data-residency requirement for a PSU (on-prem vs government cloud vs public cloud)

Happy to go deeper on any single piece next — e.g. the actual API endpoint list, or wireframes for the advance/attendance dashboards.

Nicsi vendor hrms design
Document · MD

Download

📄 🔍 🗑️ 🔄



Want to be notified when Claude responds?

Notify ×

Write a message...